

Polynomial Regression on Auto MPG Dataset with Comparison to Linear Regression

Aim

To implement **Polynomial Regression** on the Auto MPG dataset to predict **MPG using engine displacement**, and compare it with **Linear Regression** using MSE and R^2 .

Theory

◆ Linear Regression

Fits a straight-line relationship between input and output.

◆ Polynomial Regression

Extends linear regression by adding higher-degree terms to model non-linear relationships.

◆ Evaluation Metrics

- **MSE** → Lower is better
- **R^2** → Closer to 1 is better

Algorithm

1. Load dataset
2. Handle missing values
3. Select feature (displacement) and target (mpg)
4. Split dataset
5. Train linear model
6. Train polynomial models (degree 2, 3, 5)
7. Evaluate using MSE and R^2
8. Plot regression curves

Program (Error-Free Version)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import PolynomialFeatures

from sklearn.metrics import mean_squared_error, r2_score


# -----

# Load dataset

# -----

df = sns.load_dataset('mpg')


# -----

# Preprocessing

# -----

df['horsepower'] = df['horsepower'].fillna(df['horsepower'].median())

df = df.drop('name', axis=1)


# Feature and target

X = df[['displacement']] # DataFrame (important!)

y = df['mpg']


# -----

# Train-Test Split

# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# -----

```

```

# Linear Regression

# -----

lin_model = LinearRegression()

lin_model.fit(X_train, y_train)


y_pred_lin = lin_model.predict(X_test)


mse_lin = mean_squared_error(y_test, y_pred_lin)
r2_lin = r2_score(y_test, y_pred_lin)


print("Linear Regression:")
print("MSE:", round(mse_lin, 4))
print("R2:", round(r2_lin, 4))


# -----

# Polynomial Regression

# -----

degrees = [2, 3, 5]

models = {}


print("\nPolynomial Regression:")
print("Degree\tMSE\t\tR2")
print("-----")


for degree in degrees:

    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)

```

```

X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

y_pred = model.predict(X_test_poly)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"{degree}\t{mse:.4f}\t{r2:.4f}")

models[degree] = (model, poly)

# -----
# Visualization
# -----

plt.figure(figsize=(8, 6))

# Scatter plot
plt.scatter(X, y, color='blue', s=20, label='Data')

# Create smooth range (DataFrame to avoid warning)
X_range = pd.DataFrame(
    np.linspace(X['displacement'].min(), X['displacement'].max(), 100),
    columns=['displacement']
)

```

```
# Linear regression line

y_range_lin = lin_model.predict(X_range)

plt.plot(X_range, y_range_lin, color='red', label='Linear')


# Polynomial curves

colors = ['yellow', 'green', 'orange']

for degree, color in zip(degrees, colors):

    model, poly = models[degree]

    X_range_poly = poly.transform(X_range)

    y_range = model.predict(X_range_poly)

    plt.plot(X_range, y_range, color=color, label=f'Degree {degree}')

plt.xlabel("Displacement")

plt.ylabel("MPG")

plt.title("Polynomial vs Linear Regression (Error-Free)")

plt.legend()

plt.show()
```

Output

- MSE and R^2 for:
 - Linear Regression
 - Polynomial Regression (degree 2, 3, 5)
- Smooth plot showing:
 - Linear fit
 - Polynomial curves

Linear Regression:

MSE: 18.1025

R^2 : 0.6633

Polynomial Regression:

Degree	MSE	R ²

2	15.1074	0.7190
3	14.9436	0.7221
5	15.2308	0.7167